

Algorithms

Lecture # 02

Syed Hamed Raza

Example: 2-dimension maxima

Example: 2-dimension maxima

Let us do an example that illustrates how we analyze algorithms.

- Suppose you want to buy a car.
- You want to pick the fastest car.
- But fast cars are expensive; you want the cheapest.
- You cannot decide which is more important: speed or price.

Example: 2-dimension maxima

Example: 2-dimension maxima

- Definitely *do not* want a car if there is another that is both faster and cheaper.
- We say that the fast, cheap car *dominates* the slow, expensive car relative to your selection criteria.
- So, given a collection of cars, we want to list those cars that are not dominated by any other.

Example: 2-dimension maxima

Example: 2-dimension maxima

Here is how we might model this as a formal problem.

- Let a point p in 2-dimensional space be given by its integer coordinates, $p = (p.x, p.y)$.
- A point p is said to be ***dominated*** by point q if $p.x \leq q.x$ and $p.y \leq q.y$.

Example: 2-dimension maxima

Example: 2-dimension maxima

- Given a set of n points, $P = \{p_1, p_2, \dots, p_n\}$ in 2-space a point is said to be ***maximal*** if it is not dominated by any other point in P .

Example: 2-dimension maxima

Example: 2-dimension maxima

The car selection problem can be modelled this way:

For each car we associate (x, y) pair where

- x is the speed of the car and
- y is the negation of the price.

Example: 2-dimension maxima

Example: 2-dimension maxima

- High y value means a cheap car and low y means expensive car.
- Think of y as the money left in your pocket after you have paid for the car.
- Maximal points correspond to the fastest and cheapest cars.

Example: 2-dimension maxima

Example: 2-dimension maxima

2-dimensional Maxima:

- Given a set of points $P = \{p_1, p_2, \dots, p_n\}$ in 2-space,
- output the set of maximal points of P ,
- i.e., those points p_i such that p_i is not dominated by any other point of P .

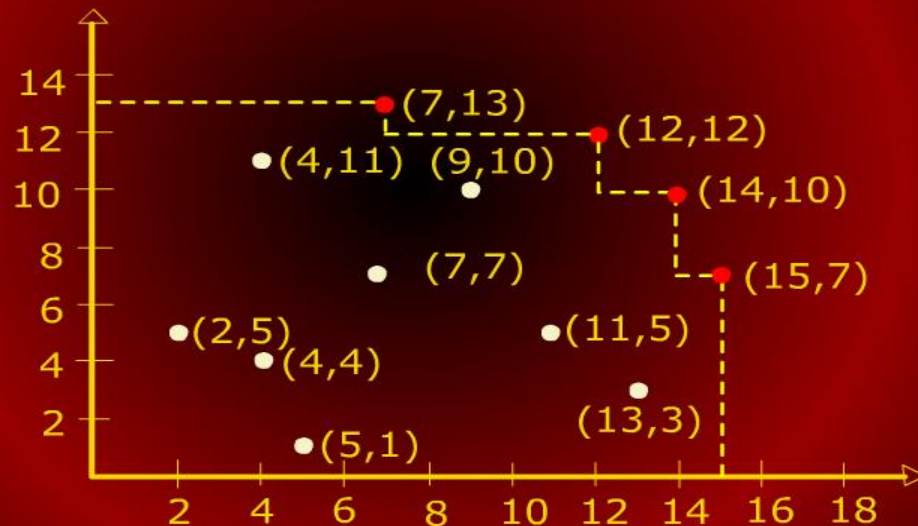
Example: 2-dimension maxima

Example: 2-dimension maxima

- Our description of the problem is at a fairly mathematical level.
- We have intentionally not discussed how the points are represented.
- We have not discussed any input or output formats.
- We have avoided programming and other software issues.

Example: 2-dimension maxima

Example: 2-dimension maxima



Example: 2-dimension maxima

Example: 2-dimension maxima

Brute-Force Algorithm:

- Let $P = \{p_1, p_2, \dots, p_n\}$ be the initial set of points.
- For each point p_i , test it against all other points p_j .
- If p_i is not dominated by any other point, then output it.

Example: 2-dimension maxima

Example: 2-dimension maxima

```
MAXIMA(int n, Point P[1 . . n])  
1  for i ← 1 to n  
2  do maximal ← true  
3    for j ← 1 to n  
4    do  
5      if (i ≠ j) and (P[i].x ≤ P[j].x) and  
        (P[i].y ≤ P[j].y)  
6      then maximal ← false; break
```

Example: 2-dimension maxima

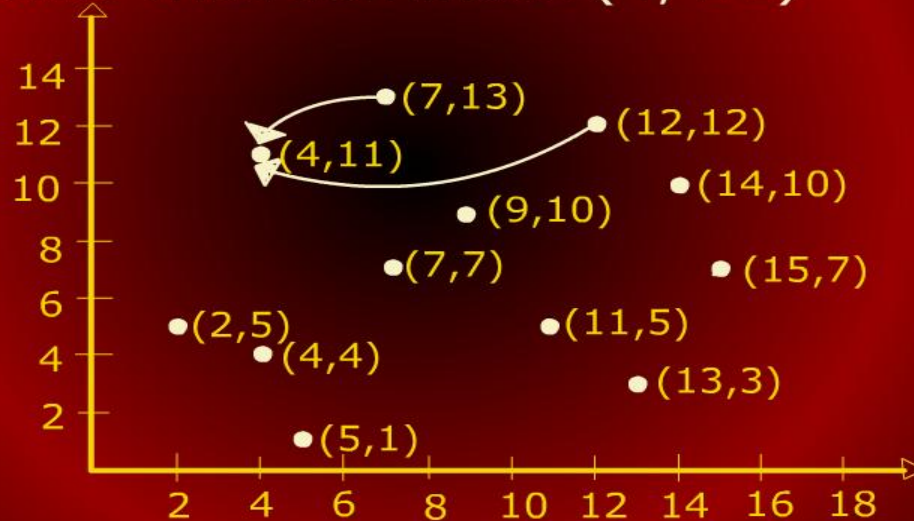
Example: 2-dimension maxima

```
3   for  $j \leftarrow 1$  to  $n$ 
4   do
5       if  $(i \neq j)$  and  $(P[i].x \leq P[j].x)$  and
         $(P[i].y \leq P[j].y)$ 
6           then  $maximal \leftarrow false$ ; break
7   if  $(maximal = true)$ 
8       then output  $P[i]$ 
```


Example: 2-dimension maxima

Example: 2-dimension maxima

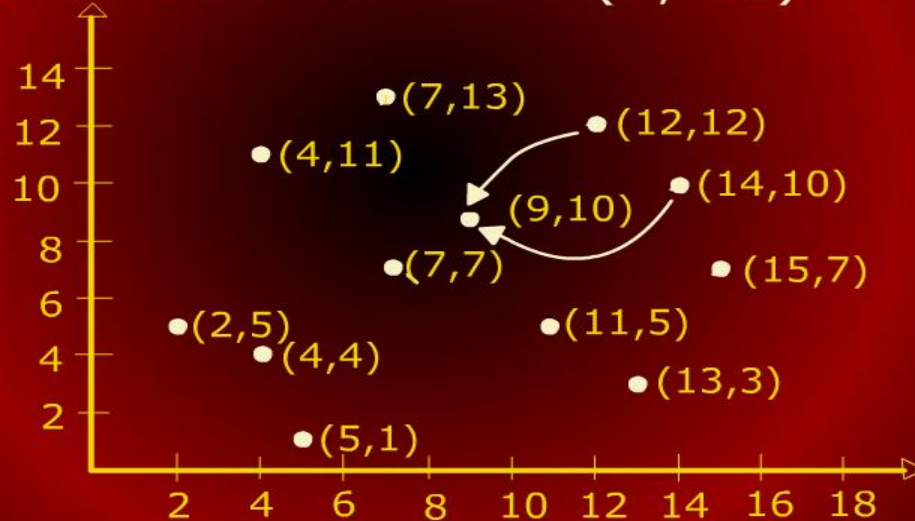
Points that dominate (4, 11)



Example: 2-dimension maxima

Example: 2-dimension maxima

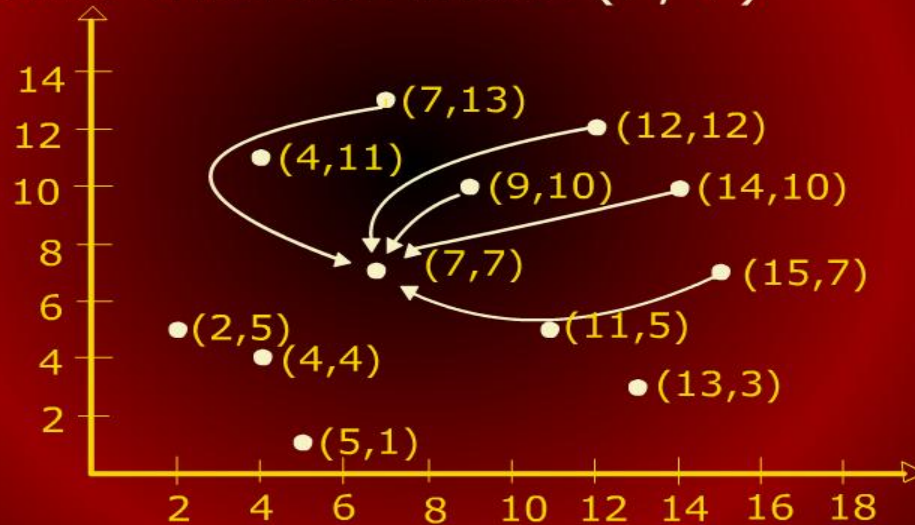
Points that dominate (9, 10)



Example: 2-dimension maxima

Example: 2-dimension maxima

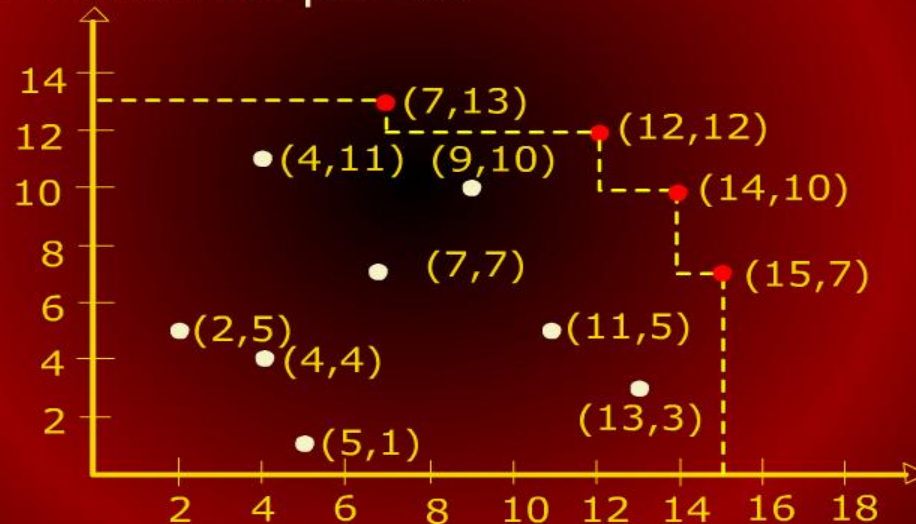
Points that dominate (7, 7)



Example: 2-dimension maxima

Example: 2-dimension maxima

The maximal points



Running Time Analysis

Running Time Analysis

Running Time Analysis

- The main purpose of our mathematical analysis will be measuring the execution time.
- We will also be concerned about the space (memory) required by the algorithm.

Running Time Analysis

Running Time Analysis

- The running time of an implementation of the algorithm would depend upon
 - the speed of the computer,
 - programming language,
 - optimization by the compiler etc.
- Although important, we will ignore these technological issues in our analysis.

Analysis of 2-d Maxima

Analysis of 2-d Maxima

To measure the running time of the brute-force 2-d maxima algorithm, we could

- count the number of steps of the pseudo code that are executed
- or, count the number of times an element of P is accessed
- or, the number of comparisons that are performed.